Arduino - Accéléromètre

1 Introduction

Un accéléromètre est un petit composant électronique permettant de mesurer l'accélération. Il peut être très utile dans des montages nécessitant une stabilité, comme un drone. Il peut également être utilisé pour détecter des chocs et bien d'autres événements.

Dans notre cas, nous allons utiliser l'ADXL327. C'est un accéléromètre de type MEMS sur trois axes : X, Y et Z. Il fonctionne avec une tension de 3,3 V.

Attention : La puce n'accepte qu'une plage de tension allant de 1,8 à 3,6 V. L'alimenter en 5 V causerait des dommages irréversibles.

2 Explication Physique

Un capteur MEMS (Micro-Electro-Mechanical Systems) est composé d'une structure mobile maintenue par des ressorts en silicium. Cette dernière possède des dents, tout comme la structure fixe. Lorsque le capteur n'est pas en mouvement, les dents mobiles se trouvent pile au milieu des dents fixes. Il y a donc un équilibre des charges.

Mais dès que l'on bouge le capteur, à cause de l'accélération, la partie mobile va se mettre à bouger et donc se rapprocher plus d'un côté que de l'autre. Ceci va créer un déséquilibre de charge. Ce dernier sera par la suite amplifié. Il y a trois capteurs MEMS par composant, un pour chaque axe.

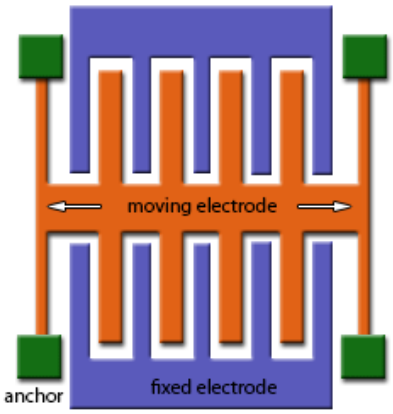

Figure 1 – Représentation d'une structure MEMS

3 Calculs

Dans un premier temps, il est nécessaire de calculer la tension de sortie de l'ADC.

$$V_{Out} = \text{Valeur binaire} \cdot \frac{V_{CC}}{4095} \tag{1}$$

Il faut ensuite calculer l'accélération.

$$\text{Accélération (g)} = \frac{\text{Tension mesurée (V)} - \text{Tension au repos (V)}}{\text{Volts par g}} \tag{2}$$

La fiche technique indique que la tension par g est de 420 mV. De plus, la tension de repos peut être facilement calculée, elle est égale à $\frac{V_{CC}}{2}$. En fonction du résultat, on peut déterminer l'inclinaison.

- Si le résultat est nul, le capteur est au repos.
- Si le résultat est positif, il est incliné dans un sens.
- Si le résultat est négatif, il est incliné dans l'autre sens.

Il ne reste plus qu'à transformer ces valeurs d'accélération en angle.

$$\theta = \text{atan2}(A_y, A_z) \cdot \frac{180}{\pi} \tag{3}$$

$$\psi = \text{atan2}(-A_x, \sqrt{A_y^2 + A_z^2}) \cdot \frac{180}{\pi} \tag{4}$$

Note : Il y a toujours une accélération due à la gravité, même lorsque l'objet n'est pas en mouvement. C'est cela qui nous permet de mesurer un angle même au repos.

4 Pinout

À l'origine, la puce est en boîtier QFN (Quad Flat No-leads). Il est donc essentiel de la souder sur un module dit « breakout » pour pouvoir l'utiliser sur une platine d'essai.

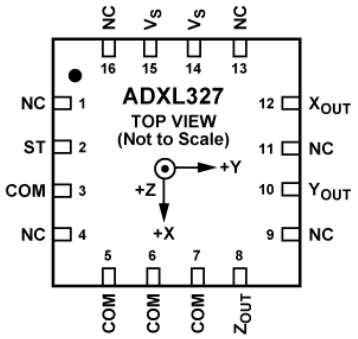

FIGURE 2 – Pinout de l'ADXL327

5 Branchement

Il est conseillé de relier toutes les alimentations à toutes les masses. Cependant, afin de simplifier le montage lors de tests, nous pouvons n'en connecter qu'une seule. Il est également nécessaire de placer un condensateur de découplage de 100 nF entre l'alimentation et la masse.

Les broches X_{Out}, Y_{Out} et Z_{Out} doivent être reliées aux entrées analogiques de la carte. De plus, il est nécessaire de placer un condensateur de $0,047\,\mu$F vers la masse sur chacune des sorties.

> **Attention :** Il faut alimenter le composant en 3,3 V et non en 5 V au risque de l'endommager.

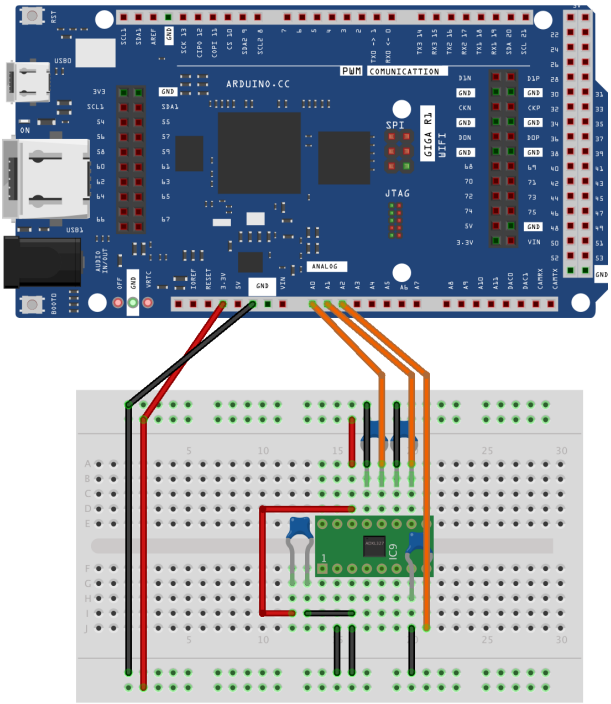

FIGURE 3 – Branchement à l'Arduino

6 Programme

```
// On change le nom pour éviter le conflit avec les fichiers système du Giga
const int RESOLUTION_ADC = 12;
const float VCC = 3.3;
const float ZERO_G = 1.65;
const float SENSITIVITY = 0.420;

void setup() {
  // On passe l'ADC à 12 bits, de base il est à 10 pour assurer la
  ↪  compatibilité
  // On peut le pousser jusqu'à 16 bits, mais ce n'est pas très utile
  analogReadResolution(RESOLUTION_ADC);

  // On démarre la communication série
```

```
  Serial.begin(9600);
}

void loop() {
  // On lit les valeurs brutes (sur 12 bits)
  int rawX = analogRead(A0);
  int rawY = analogRead(A1);
  int rawZ = analogRead(A2);

  // On convertit en tension
  float volX = rawX * (VCC / 4095.0);
  float volY = rawY * (VCC / 4095.0);
  float volZ = rawZ * (VCC / 4095.0);

  // On convertit les tensions en force
  float accX = (volX - ZERO_G) / SENSITIVITY;
  float accY = (volY - ZERO_G) / SENSITIVITY;
  float accZ = (volZ - ZERO_G) / SENSITIVITY;

  // Calcul des angles en degrés avec la trigonométrie
  float inclinaisonAxeX = atan2(accY, accZ) * 180.0 / PI;
  float inclinaisonAxeY = atan2(-accX, sqrt(accY * accY + accZ * accZ)) *
  ↪  180.0 / PI;

  // On affiche les résultats dans le moniteur série
  Serial.print("Inclinaison (X): "); Serial.print(inclinaisonAxeX);
  ↪  Serial.print("° | ");
  Serial.print("Inclinaison (Y): "); Serial.print(inclinaisonAxeY);
  ↪  Serial.println("°");

  // On effectue un délai pour laisser le moniteur série lisible
  delay(1000);
}
```